PULSEGATE

Project Flow Guide: how every request moves through the system

Problem statement, what makes it unique, architecture, each flow step by step with the file that handles it, all five rate-limiting algorithms with their logic, the complete business rules, how Docker runs the database, and why each design decision was made.

Project	Intelligent API Traffic Gateway	**Style**	Modular monolith
Backend	Java 21 · Spring Boot 4 · JPA	**Frontend**	React 19 · Vite · Tailwind · Recharts
Database	PostgreSQL 16 (Docker)	**Algorithms**	Token Bucket · Fixed Window
Tests	74 automated · 95.8% line coverage	**Repository**	github.com/NehaPawar65/PulseGate

1. Problem Statement

A public API can receive **too many requests from a single client** in a short time. Typical causes:

- a buggy app stuck in a retry loop
- a script scraping data
- one large customer using all the capacity
- a deliberate flood of requests

Without protection, **every** user sees slow responses or errors, and server costs rise, because of one caller.

> **Solution.** PulseGate sits in front of the API. For every request it **identifies** the caller by API key, **checks** the caller's traffic policy, **allows** the request or rejects it with **HTTP 429 Too Many Requests** plus a `Retry-After` time, and **records** the decision for analytics.

2. Why This Is Useful

Who	Benefit
API owner	One noisy client cannot take the service down; load and cost stay predictable.
Product / pricing	Fair-use tiers: Free = 10 req/s, Pro = 100 req/s, each a separate policy.
Well-behaved clients	`Retry-After` tells them exactly when to retry, instead of guessing or hammering.
Operations team	The dashboard shows who is being limited, how often, and how fast the gateway is.
Security	Slows brute-force and credential-stuffing attempts; keys are stored only as hashes.

Real systems built on the same idea: GitHub API (5,000 requests per hour), Stripe, AWS API Gateway, Cloudflare and Nginx `limit_req`.

3. What Makes PulseGate Unique & Important

Why rate limiting matters

Area	Why it matters
Reliability	One runaway client is the most common way a healthy API gets overloaded. A gateway contains the damage to that one client.
Cost control	Every request costs compute, database and bandwidth. Limits keep the bill tied to what each customer pays for.
Security	Rate limits slow brute-force logins, credential stuffing, scraping and simple denial-of-service floods.
Business model	API products are sold in tiers (Free / Pro / Enterprise). The policy *is* the product plan, enforced in code.
Interviews	"Design a rate limiter" is one of the most frequently asked system-design questions. This project is a working, measured answer to it.

What sets this project apart

1. Measured, not just claimed

A built-in simulator sends **real concurrent HTTP traffic** through the full gateway path. The results match the maths: 10 + 10 × 2.22 s ≈ 32 predicted, **32 allowed**.

2. Two algorithms, compared side by side

Token Bucket and Fixed Window sit behind one `RateLimiter` interface and are compared on the **same workload**. The difference (smooth vs bursty) is visible in the charts.

3. Proven correct under concurrency

It handles the race most implementations miss: two *first* requests each creating a full bucket. `ConcurrentHashMap.compute()` prevents it, and multi-threaded tests prove it (10 of 10 repeated runs passed).

4. Fails safe

The limit decision is made in memory, before anything is written. If the analytics database fails, **limits still hold**; only an analytics row is lost.

5. Security built in

192-bit random API keys, **shown once** and stored only as SHA-256 hashes. Keys never appear in logs or list responses.

6. Live configuration

Editing a policy takes effect on the client's **next request**, with no restart or redeploy. The limiter notices its settings changed and swaps itself.

7. Full stack with observability

Not just an algorithm: REST APIs, PostgreSQL history, a React dashboard with live charts, latency percentiles and a filterable request log.

8. Honest engineering

74 tests, 95.8% line coverage, real numbers only, and documented limitations with a clear path to scale (Redis, caching, batching).

In one line: most portfolio projects show that a developer can build CRUD screens. PulseGate shows **algorithms, concurrency, security, system design and measurement** working together in one running system.

4. Architecture

React Dashboard :5173

Dashboard · Clients & Policies
Simulator · Request Log
`services/api.js → fetch`

Any API client

curl · mobile app · simulator
`header X-API-Key: pg_…`

→

Spring Boot :8080

Admin APIs: policies · clients · analytics · events · simulations
RateLimitInterceptor: guards `/api/protected/**`
RateLimiterService: one limiter per client, *in memory*
ProtectedController: the sample API being protected
RequestEventRecorder: stores every decision

↓

PostgreSQL :55432

`rate_limit_policies` · `clients` · `request_events`

> **Key design choice:** live quota state (tokens) is kept **in memory** because it is checked on every request and must be fast. Policies, clients and history go in **PostgreSQL** because they must survive a restart.

The five flows in this guide

Flow	Name	What happens
A	Setup: policy, client, API key	Admin defines a rule and issues a key (stored hashed).
B	Request through the gateway	Identify, then limit, then 200/429/401, then record. **The core of the project.**
C	Rate-limit algorithms	Token Bucket and Fixed Window, thread-safe and O(1).
D	Traffic simulator	Concurrent real HTTP load through Flow B, with results measured.
E	Analytics & dashboard	Aggregates computed in SQL, shown as live charts.

5. Flow A: Setup (Policy → Client → API Key)

1 **Admin fills the "New policy" form**

`frontend/src/pages/ClientsPage.jsx` PolicyForm checks the fields in the browser, then calls `api.createPolicy()` in `services/api.js` .

2 **POST /api/policies: the request body is validated**

`controller/PolicyController.java` `dto/CreatePolicyRequest.java`

`@Valid` enforces capacity 1–100,000, refill ≤ 100,000 and window 1–86,400 s. Bad input → **400**

3 **Business rules, then save**

`service/PolicyService.java` `entity/RateLimitPolicy.java`

Token Bucket requires refill > 0, and Fixed Window requires `windowSeconds`. A duplicate name → **409**. Otherwise an INSERT into `rate_limit_policies`.

4 **Register a client: POST /api/clients {name, policyId}**

`controller/ClientController.java` `service/ClientService.java` `service/ApiKeyService.java`

Generate the key `"pg_" + 24 random bytes` (192 bits), then compute its **SHA-256 hash**. Save `Client(name, hash, first 8 chars, policy)`. **The raw key is never stored.**

5 **The key is shown exactly once**

The response `{client, apiKey}` is the only time the full key exists outside the client. The **KeyRevealDialog** shows it with a Copy button; `utils/keyStore.js` remembers it in the browser for the simulator (a demo convenience).

Why hash the key?

If the database leaks, the attacker gets hashes, which can't be used to call the API. GitHub and Stripe handle tokens the same way.

Why SHA-256, not bcrypt?

bcrypt is deliberately slow and salted to protect guessable passwords. A 192-bit random key can't be guessed, so a fast, deterministic hash is safe and allows an **indexed lookup** by hash.

6. Flow B: A Request Through the Gateway (Core)

Request: `GET /api/protected/resource` with header `X-API-Key: pg_xxx`

1 **Interceptor is attached only to protected routes**

`config/WebConfig.java` registers the interceptor for `/api/protected/**`. Health and admin APIs are not rate limited. CORS for the React app is configured here too.

2 **preHandle() runs before the controller**

`limiter/RateLimitInterceptor.java`

Browser CORS preflight (`OPTIONS`) is passed through. Start a latency timer with `System.nanoTime()`, then read the `X-API-Key` header.

✗ missing / blank key → `401` "Missing X-API-Key header", recorded as `REJECTED_INVALID_KEY`

3 **Identify the client: one database query**

`repository/ClientRepository.java` `findByApiKeyHash(sha256(key))` with `@EntityGraph`, which loads the client **and** its policy together.

✗ unknown or inactive → `401` "Invalid or inactive API key", recorded as `REJECTED_INVALID_KEY`

4 **Ask the rate limiter for permission**

`limiter/LimitSpec.java` `limiter/RateLimiterService.java`

Build `LimitSpec(algorithm, capacity, refillRate, window)` from the policy, then call `tryConsume(clientId, spec)`. This is an in-memory, O(1) check.

✗ no quota left → `429` with `Retry-After`, recorded as `REJECTED_RATE_LIMIT`; the controller never runs

✓ allowed → add the headers `X-RateLimit-Limit` and `X-RateLimit-Remaining`, then continue

5 **The protected API runs**

`controller/ProtectedController.java` returns `200` `{message, timestamp}`. It contains no rate-limit code at all; the interceptor handles that.

6

afterCompletion() records the decision

`service/RequestEventRecorder.java` saves `RequestEvent(clientId, path, ALLOWED, 200, latency, remaining)` to `request_events`.

How rejections become proper HTTP responses

The interceptor **throws** `InvalidApiKeyException` or `RateLimitExceededException`. `exception/GlobalExceptionHandler.java` catches them and returns standard RFC 9457 *ProblemDetail* JSON:

```
HTTP/1.1 429
Retry-After: 1
X-RateLimit-Limit: 5
X-RateLimit-Remaining: 0

{ "status": 429, "title": "Too Many Requests",
  "detail": "Rate limit of 5 requests exceeded. Retry after 1s.", "retryAfterSeconds": 1 }
```

Failure behaviour (key interview point). The limit decision is made **in memory, before** anything is written. If saving the event fails (for example, the database is down), `RequestEventRecorder` catches the error, logs a warning, and the response is unchanged. We lose one analytics row, but **limits are never bypassed** because of a database problem. `RequestEventRecorderTest` checks this.

7. Flow C: The Rate-Limit Algorithms

7.1 One limiter per client: RateLimiterService.java

```
ConcurrentHashMap<Long clientId, Entry(spec, limiter)> limiters;

Entry e = limiters.compute(clientId, (id, existing) ->
        existing != null && existing.spec().equals(spec)
            ? existing                                  // same policy: reuse
            : new Entry(spec, spec.newLimiter()));    // first request or policy changed
return e.limiter().tryConsume();                        // outside compute(): short lock
```

- **Atomic creation.** If two first requests from a new client arrive together, only one limiter is created. A plain "check, then put" would let each create its own full bucket, giving the client double the quota.
- **Live policy changes.** When a policy is edited, the spec no longer matches, so a fresh limiter is swapped in on the next request with no restart.

7.2 Token Bucket: TokenBucket.java

A bucket holds up to *capacity* tokens and refills at *rate* tokens per second. Each request takes one token.

```
state: tokens, lastRefillNanos
synchronized tryConsume():
  elapsed = now - lastRefill
  tokens = min(cap, tokens + elapsed*rate)
  if tokens >= 1: tokens -= 1   → ALLOW
  else → REJECT, retry = (1-tokens)/rate
```

Example (capacity 5, rate 1/s): 5 instant requests are allowed (the burst), the 6th gets a 429 with `Retry-After: 1`, and after 1 s one more is allowed.

7.3 Fixed Window: FixedWindowCounter.java

At most *capacity* requests per window; the count resets when a new window starts.

```
state: windowStart, count
synchronized tryConsume():
  if now past window end:
     move to current window; count = 0
  if count < cap: count++   → ALLOW
  else → REJECT, retry = window end - now
```

Example (10 per 1 s): 10 requests pass at the start of each second, then nothing until the next window.

7.4 Why it is O(1), thread-safe and clock-safe

Property	How
O(1) time and memory	Two numbers per client and a few arithmetic steps per request. No timer thread, no list of past requests.
No token spent twice	`synchronized` makes refill-then-take atomic. A test runs 50 threads making 5,000 attempts against 1,000 tokens and asserts exactly 1,000 succeed (passed 10 of 10 repeated runs).
Clock safety	`System.nanoTime()` only moves forward. The wall clock can jump backwards during NTP syncs.
Clean layering	The limiter takes plain values (`LimitSpec`), not database entities. It can be unit-tested without Spring, or moved to Redis later.

7.5 Measured comparison: same average rate (10 req/s), same workload

Workload: 2,000 requests at concurrency 10. Allowed requests per 100 ms slice:

Token Bucket: burst, then smooth



Traffic got through in **16 of 16** slices.

Fixed Window: clumps



Traffic got through in only **2 of 17** slices.

Fixed Window also allows up to **2× capacity across a window boundary**: a full quota at 0.9 s and another at 1.1 s. Token Bucket allows only 6 in that same scenario (verified by `FixedWindowCounterTest`). This is why most gateways prefer Token Bucket.

8. Rate-Limiting Algorithms: Logic, Code & Comparison

There are five standard rate-limiting algorithms. PulseGate implements the first two, **Token Bucket** and **Fixed Window**, behind one `RateLimiter` interface. The other three are explained so you can compare them and justify the choice in interviews.

Algorithm	State per client	Bursts?	In PulseGate
1. Token Bucket	tokens, last refill time	Yes, up to capacity	✓ **Implemented**: `TokenBucket.java` (default)
2. Fixed Window Counter	window start, count	Yes, up to 2× at a boundary	✓ **Implemented**: `FixedWindowCounter.java`
3. Leaky Bucket	queue (or water level), last leak time	No, constant output	Explained (not implemented)
4. Sliding Window Log	list of request timestamps	No, exact	Explained (not implemented)
5. Sliding Window Counter	current + previous window counts	Very small	Explained (a planned next step)

8.1 Token Bucket (implemented)

Idea. Each client has a bucket holding up to *capacity* tokens. Tokens drip in at a steady *refill rate*. Every request must take one token; with no token, the request is rejected.

Logic, step by step:

- **1. Refill lazily.** Work out the time since the last call and add `elapsed × rate` tokens, without going above capacity. There is no background timer.
- **2. Check.** If at least one whole token is available, take it and allow the request.
- **3. Reject.** Otherwise reject, and report how long until one token exists: `(1 − tokens) ÷ rate`.

PulseGate's actual code (`limiter/TokenBucket.java`):

```
public synchronized ConsumeResult tryConsume() {      // synchronized: one thread at a time per bucket
    refill();
    if (tokens >= 1) {
        tokens -= 1;
        return new ConsumeResult(true, (long) tokens, 0);
    }
    long waitNanos = (long) Math.ceil((1 - tokens) * NANOS_PER_SECOND / refillTokensPerSecond);
    return new ConsumeResult(false, 0, waitNanos);
}

private void refill() {
    long now = nanoClock.getAsLong();                    // System.nanoTime(): never goes backwards
    long elapsed = now - lastRefillNanos;
    if (elapsed > 0) {
        tokens = Math.min(capacity, tokens + elapsed * refillTokensPerSecond / NANOS_PER_SECOND);
        lastRefillNanos = now;
    }
}
```

Worked example (capacity 5, refill 1 token/s, the bucket starts full):

Time	Event	Tokens after	Result
0.0 s	5 requests at once	5 → 0	5 × `200`
0.0 s	6th request	0	`429`, Retry-After 1 s
0.5 s	request	0.5 (not a whole token)	`429`, Retry-After 1 s (0.5 s rounded up)
1.0 s	request	1.0 → 0	`200`
60 s idle	tokens refill	capped at 5	the next burst can be 5, never 60

Pros: allows short bursts (good for real users); smooth long-term rate; O(1) time and memory; two settings that are easy to explain (burst size and steady rate).

Cons: two numbers to tune; in a distributed setup the refill-and-take step must be atomic (for example, a Redis Lua script). **Used by:** AWS API Gateway, Stripe.

8.2 Fixed Window Counter (implemented)

Idea. Split time into fixed windows (for example, each 60 seconds). Count requests in the current window; allow up to *capacity*, then reject until the next window begins.

Logic, step by step:

- **1. Roll the window.** If the current window has ended, jump to the window containing "now" and reset the count to 0.
- **2. Check.** If `count < capacity`, increment and allow.
- **3. Reject.** Otherwise reject; retry after `windowStart + windowLength − now`.

PulseGate's actual code (`limiter/FixedWindowCounter.java`):

```
public synchronized ConsumeResult tryConsume() {
    long now = rollWindow();
    if (count < capacity) {
        count++;
        return new ConsumeResult(true, capacity - count, 0);
    }
    return new ConsumeResult(false, 0, windowStartNanos + windowNanos - now);
}

private long rollWindow() {                              // keeps windows aligned
    long now = nanoClock.getAsLong();
    long elapsed = now - windowStartNanos;
    if (elapsed >= windowNanos) {
        windowStartNanos = now - (elapsed % windowNanos);
        count = 0;
    }
    return now;
}
```

Worked example and the boundary problem (capacity 5 per 1 s window):

Time	Window	Count	Result
0.9 s	window 1 (0–1 s)	0 → 5	5 × `200`
1.1 s	window 2 (1–2 s): reset	0 → 5	5 × `200`

10 requests in 0.2 s, twice the intended rate. A Token Bucket with the same average rate allows only 6 here; `FixedWindowCounterTest` proves both numbers.

Pros: the simplest to build and explain; O(1); easy to share in Redis (`INCR` + `EXPIRE`); matches "N per hour" plans that reset at a fixed time.

Cons: up to 2× bursts at window edges; traffic arrives in clumps (measured: 10, then 9 empty slices, then 10); clients all retry at the window reset at once.

8.3 Leaky Bucket (explained)

Idea. Requests pour into a bucket that **leaks at a constant rate**. If the bucket is full, new requests overflow and are rejected. The output is perfectly smooth.

```
state: level (or a queue), lastLeakTime
on request:
    level = max(0, level - (now - lastLeakTime) * leakRate)   // drain since last time
    lastLeakTime = now
    if level + 1 <= capacity: level += 1 → ALLOW (or enqueue)
    else → REJECT                                              // bucket overflows
```

Example: leak rate 2/s, capacity 4. A burst of 10 → 4 accepted and processed one every 0.5 s; 6 rejected.

Pros: constant, predictable output rate, which is ideal for protecting a fragile downstream system; O(1).

Cons: no bursts, even for idle clients; the queueing version adds waiting time. **Used by:** Nginx `limit_req` and traffic shaping in networks.

8.4 Sliding Window Log (explained)

Idea. Store the **timestamp of every request**. A request is allowed if fewer than *limit* timestamps fall inside the last *window* seconds, measured back from now.

```
state: deque of timestamps
on request:
    while log.first <= now - window: log.removeFirst()      // drop old entries
    if log.size < limit: log.addLast(now) → ALLOW
    else → REJECT, retry after (log.first + window - now)
```

Example: limit 3 per 10 s; requests at 1 s, 2 s, 3 s are allowed; a request at 5 s is rejected; at 11.1 s the 1 s entry expires, so it is allowed.

Pros: exact: never more than *limit* requests in any window, with no boundary problem.

Cons: memory grows with the limit (O(limit) per client) and each check does more work; expensive for large limits such as 10,000 per hour.

8.5 Sliding Window Counter (explained)

Idea. A cheap approximation of the sliding log. Keep counts for the **current** and **previous** fixed windows, and weight the previous one by how much of it still overlaps the sliding window.

```
estimated = currentCount + previousCount × (1 - elapsedInCurrentWindow / window)
if estimated < limit: currentCount += 1 → ALLOW   else → REJECT
```

Example: limit 10 per minute. The previous minute had 8, the current minute has 3, and we are 15 s (25%) into it: `3 + 8 × 0.75 = 9` → allowed. One more request makes the estimate 10 → rejected.

Pros: O(1) memory (two counters); almost no boundary burst; easy in Redis. **Used by:** Cloudflare.

Cons: approximate, because it assumes requests were spread evenly across the previous window.

8.6 Side-by-side comparison

Property	Token Bucket	Fixed Window	Leaky Bucket	Sliding Log	Sliding Counter
Accuracy	Exact (rate + burst)	2× at edges	Exact	**Exact**	Approximate
Memory per client	O(1)	O(1)	O(1)*	O(limit)	O(1)
Time per request	O(1)	O(1)	O(1)	O(expired)	O(1)
Allows bursts	**Yes, bounded**	Yes, unbounded at edges	No	No	Slightly
Output shape	Burst, then smooth	Clumps per window	Constant	Even	Nearly even
Simplicity	Medium	**Highest**	Medium	Medium	Medium
Best for	Public APIs	Simple quotas / billing	Protecting a fragile backend	Strict, low limits	Large-scale, distributed

* The queueing version of Leaky Bucket stores up to *capacity* waiting requests.

8.7 Why PulseGate uses Token Bucket as the default

> Real API users are bursty: an app opens and fires several calls at once, then goes quiet. **Token Bucket allows that bounded burst and still enforces the long-term rate**, using O(1) memory and time. Fixed Window was added as a contrast, and PulseGate's simulator **measured** the difference at the same 10 req/s average: Token Bucket let traffic through in 16 of 16 time slices, Fixed Window in only 2 of 17. The natural next algorithm is the **Sliding Window Counter**, which removes Fixed Window's edge burst at the same O(1) cost.

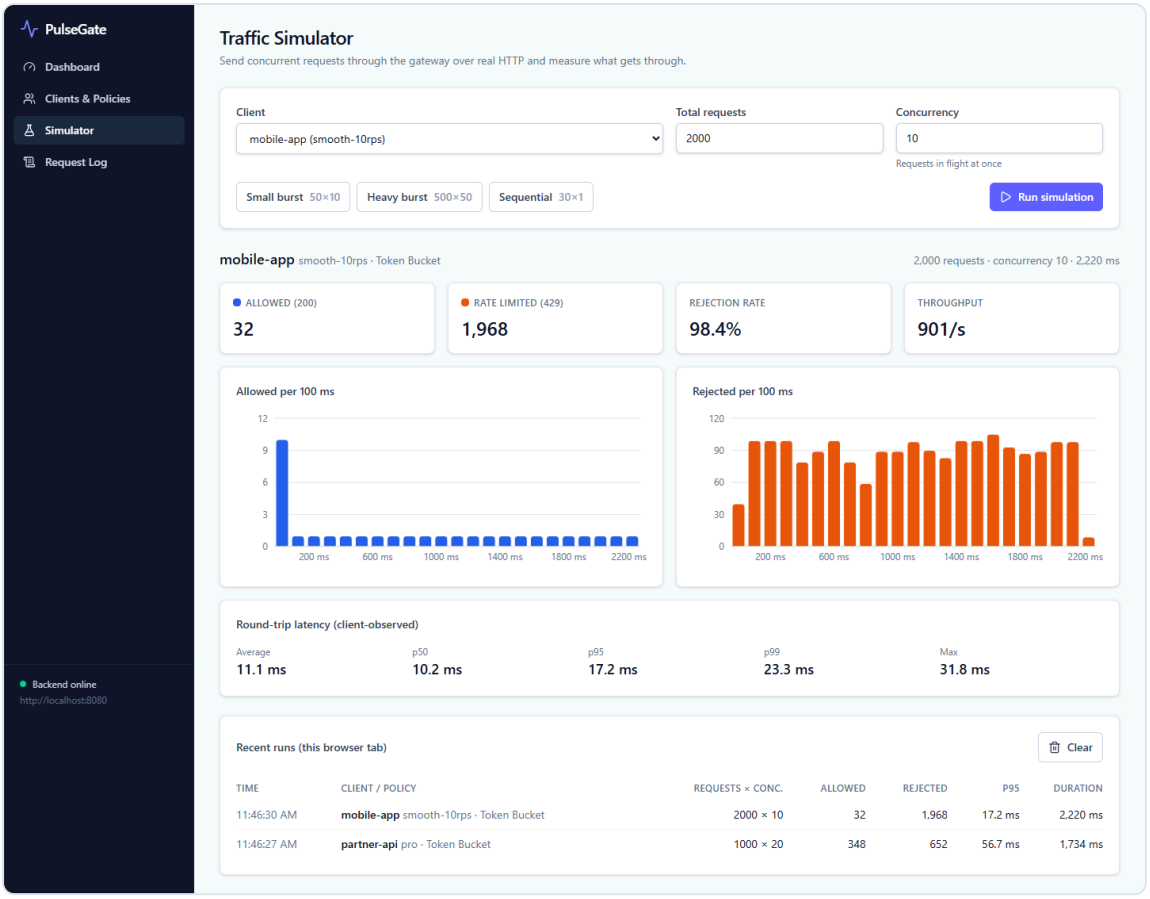
9. Flow D: Traffic Simulator

1. **User picks a client and a workload**
   `pages/SimulatorPage.jsx` lets you pick a client (its key comes from `keyStore.js`), total requests (≤ 5,000), concurrency (≤ 100), or a preset.

2. **POST /api/simulations**
   `controller/SimulationController.java` `dto/SimulationRequest.java`
   The target is always the server's **own** `http://localhost:<port>/api/protected/resource`, never a URL from the caller, so it can't be used to attack other servers.

3. **Closed-loop load generator**
   `service/SimulationService.java` creates a thread pool of *concurrency* workers. Each worker takes the next request number, sends **real HTTP**, records the status and latency, and repeats. This keeps exactly *concurrency* requests in flight, and every request passes through **Flow B**.

4. **Results returned and drawn**
   `dto/SimulationResult.java` returns allowed / rate-limited / unauthorized / errors, p50/p95/p99 latency, throughput, and a timeline of allowed and rejected per 100 ms. The UI shows stat cards, two charts and a **recent runs** table for comparing algorithms.



Simulator, Token Bucket client: 32 allowed out of 2,000. A burst of 10, then 1 per 100 ms at the refill rate.

10. Flow E: Analytics & Dashboard

1

Dashboard requests a summary every 5 seconds

`pages/DashboardPage.jsx` uses `hooks/useApi.js` (auto-refresh, ignores out-of-date responses) to call `GET /api/analytics/summary`.

2

Aggregation runs inside PostgreSQL

`service/AnalyticsService.java` `repository/RequestEventRepository.java`

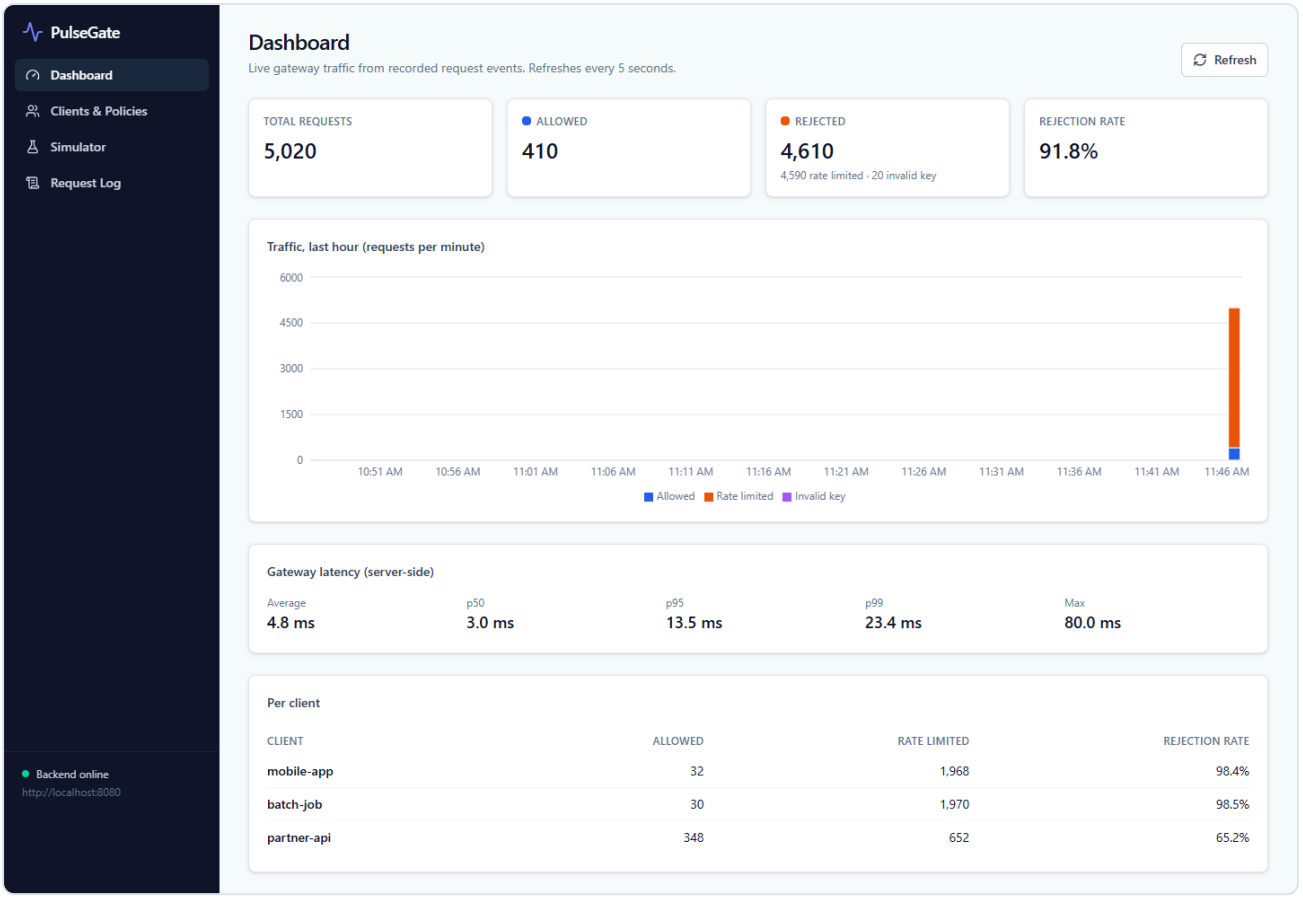
- `countGroupedByDecision()`: totals of allowed, rate-limited and invalid-key requests
- `latencyStats()`: `percentile_cont` for p50, p95 and p99
- `countGroupedByClientAndDecision()`: rejection rate per client
- `countPerMinuteSince(1 hour ago)`: the timeline chart

Why in SQL? With 100,000 events, Postgres returns about 10 summary rows instead of sending 100,000 rows to Java.

3

Request Log shows individual decisions

`pages/RequestLogPage.jsx` calls `GET /api/events?limit=&clientId=`. `service/EventService.java` returns them newest first, with client names fetched in **one** query. Filters for client, decision and size; **Clear history** calls `DELETE /api/events`.



Dashboard: totals, rejection rate, per-minute traffic, server-side latency percentiles, and a per-client breakdown.

11. Business Logic & Rules

These are the rules the system enforces: what is allowed, what is rejected, and how every number is calculated. Each rule lists the file that enforces it.

A. Policy rules

#	Rule	Enforced in
P1	A policy name is required, at most 50 characters, and **unique**. A duplicate returns **409 Conflict**.	`CreatePolicyRequest`, `PolicyService`
P2	Algorithm must be `TOKEN_BUCKET` or `FIXED_WINDOW`.	DTO + database check constraint
P3	Capacity must be a whole number from **1 to 100,000**.	`@Min(1) @Max(100000)`
P4	Token Bucket **requires** a refill rate > 0 (up to 100,000 tokens/s).	`PolicyService.validatedRefill`
P5	Fixed Window **requires** a window length from 1 to 86,400 seconds (one day).	`PolicyService.validatedRefill`
P6	A policy's name **cannot change** after creation; algorithm, capacity, rate and window can.	`UpdatePolicyRequest` has no name field
P7	A policy change applies on each client's **next request** and starts them on a **fresh, full quota** under the new limits.	`RateLimiterService` (spec comparison)

B. Client & API key rules

#	Rule	Enforced in
C1	A client name is required, at most 100 characters, and unique (409 if duplicated).	`CreateClientRequest`, `ClientService`
C2	Every client must belong to an **existing policy** (404 otherwise). One policy can serve many clients.	`ClientService.register`
C3	Key format is `pg_` followed by 32 URL-safe characters (192 random bits from `SecureRandom`).	`ApiKeyService.generate`
C4	Only the **SHA-256 hash** and an 8-character display prefix are stored. The full key is returned **once**, at registration.	`ClientService`, `ClientResponse`
C5	A lost key cannot be recovered; the client must be issued a new one.	by design (hash only)
C6	New clients are **active**; inactive clients are rejected exactly like unknown keys (401).	`RateLimitInterceptor`

C. Gateway decision rules (in this order)

#	Rule	Result
G1	Only `/api/protected/**` is rate limited. Health and admin APIs are not.	scope
G2	A missing **or blank** `X-API-Key` header is rejected first.	`401` Missing X-API-Key header
G3	An unknown or inactive key is rejected **before** any quota is spent.	`401` Invalid or inactive API key
G4	A known client with no quota left is rejected, and the protected API is never called.	`429` + `Retry-After`
G5	Otherwise the request is allowed, with headers `X-RateLimit-Limit` = capacity and `X-RateLimit-Remaining` = whole tokens left.	`200`
G6	Each client has its **own independent quota**; one client's traffic never uses another's.	`RateLimiterService`
G7	Every decision (allowed, rate limited, invalid key) is recorded. A recording failure **never** changes the response.	`RequestEventRecorder`

D. Quota calculation rules

#	Rule	Example
Q1	**Token Bucket starts full**, so a new client can burst up to its capacity immediately.	capacity 5 → 5 instant requests allowed
Q2	Tokens refill continuously, including fractions: `tokens = min(capacity, tokens + seconds × rate)`.	rate 2/s, 0.25 s → +0.5 token
Q3	A request needs one **whole** token; the bucket never holds more than capacity.	idle for 1 hour → still 5, not 3,600
Q4	Fixed Window allows *capacity* requests per window, then rejects until the next window. There is no partial refill.	10 per 60 s → 11th waits for the window to end
Q5	Windows stay aligned to their first window, even after idle periods.	windows 0–10, 10–20, 20–30 s
Q6	`Retry-After` is the time until one request would succeed, **rounded up** to whole seconds, and is never 0 on a rejection.	0.4 s → 1 s · 1.000001 s → 2 s
Q7	Quotas live in memory: a server restart gives every client a fresh, full quota.	documented trade-off

E. Analytics & reporting rules

#	Rule / formula	Where
A1	**Overall rejection rate** = (rate limited + invalid key) ÷ total requests.	`AnalyticsService`
A2	**Per-client rejection rate** = rate limited ÷ (allowed + rate limited). Invalid keys have no client, so they are excluded.	`AnalyticsService`
A3	Latency percentiles (p50/p95/p99) are computed by PostgreSQL over all recorded events, in microseconds.	`RequestEventRepository`
A4	The dashboard timeline covers the **last 60 minutes**, per minute; empty minutes are shown as zero.	`AnalyticsService`, `TrafficTimelineChart`
A5	The request log returns the newest events first, `limit` defaults to 50 and is clamped to 1–500.	`EventService`
A6	Clearing history deletes events only. **Quotas are not reset.**	`EventController`

F. Simulation rules

#	Rule	Where
S1	Total requests 1–5,000 and concurrency 1–100; concurrency is capped at the total.	`SimulationRequest`, `SimulationService`
S2	The target is always this server's own protected endpoint, never a caller-supplied URL (prevents abuse as an attack tool).	`SimulationController`
S3	Outcomes: 200 = allowed, 429 = rate limited, 401 = unauthorized, anything else = error. Rejection rate = (total − allowed) ÷ total.	`SimulationService`
S4	Simulated requests are real traffic: they use the client's real quota and appear in the dashboard and request log.	by design
S5	The timeline groups requests by **start time** into 100 ms slices; percentiles use the nearest-rank method.	`SimulationService`

> **Worked example.** Policy *free* = Token Bucket, capacity 5, refill 1/s. A client sends 7 requests instantly: requests 1–5 → **200** (Remaining 4, 3, 2, 1, 0), requests 6–7 → **429** with `Retry-After: 1`. Two seconds later, 2 tokens have refilled, so 2 more requests → **200**. The admin then edits *free* to capacity 50, and the client's next request → **200** with Remaining 49.

12. Frontend Structure

```
main.jsx → App.jsx            router; each page's code loads only when first opened
  → components/Layout.jsx      sidebar + "Backend online" health check every 10 s
    → pages/*.jsx              Dashboard · Clients & Policies · Simulator · Request Log
      → hooks/useApi.js        loading / error / data, auto-refresh, ignores out-of-date responses
        → services/api.js      the ONLY place that calls fetch; readable error messages
```

Folder	Contents
`pages/`	DashboardPage, ClientsPage, SimulatorPage, RequestLogPage
`components/`	`ui.jsx` (cards, buttons, loading/error/empty states), `forms.jsx`, `Modal.jsx`, `DecisionBadge.jsx`, `Layout.jsx`
`charts/`	`StackedBarChart.jsx`, `TrafficTimelineChart.jsx`, `colors.js` (blue/orange/purple, checked for colour-blind readers)
`utils/`	`format.js` (numbers, %, ms), `keyStore.js` (browser key storage for the demo)

13. Database (Three Separate Tables)

Table	Columns	Constraints
`rate_limit_policies`	name, algorithm, capacity, refill_tokens_per_second, window_seconds, created_at	Unique name; algorithm limited to TOKEN_BUCKET or FIXED_WINDOW
`clients`	name, api_key_hash, api_key_prefix, active, policy_id, created_at	Unique name and hash; foreign key to policies
`request_events`	client_id, path, decision, http_status, latency_micros, remaining_tokens, timestamp	Indexes on timestamp and client_id; client_id empty for invalid keys

`docker-compose.yml` runs PostgreSQL 16. `docker/init/01-create-test-db.sql` creates a separate `pulsegate_test` database, so tests never touch dev data.

14. Docker: How the Database Runs

PulseGate uses Docker for **one thing: running PostgreSQL**. The backend (Maven) and frontend (Vite) run directly on your machine, which keeps the edit-and-reload loop fast. Docker provides an identical PostgreSQL 16 for everyone, with no local install and a reset in one command.

14.1 The pieces

Your machine (host)

Spring Boot (`mvn spring-boot:run`) :8080

Vite (`npm run dev`) :5173

Tests (`mvn test`)

`jdbc:postgresql://localhost:55432/…`

⇄

Docker Desktop (Linux VM via WSL 2)

Container **pulsegate-db**, image `postgres:16`

Listens on `5432` inside the container

Port mapping: **host 55432 → container 5432**

Databases: `pulsegate` (dev) · `pulsegate_test` (tests)

↓

Named volume pgdata

Mounted at `/var/lib/postgresql/data`. Data survives container restarts and re-creation.

14.2 docker-compose.yml, line by line

```yaml
services:
  db:
    image: postgres:16                # official PostgreSQL 16 image from Docker Hub
    container_name: pulsegate-db      # fixed name, used in docker exec commands
    environment:
      POSTGRES_DB: pulsegate          # database created on first start
      POSTGRES_USER: pulsegate        # user the app connects as
      POSTGRES_PASSWORD: pulsegate    # local dev only, never in production
    ports:
      - "55432:5432"                  # host:container; avoids clashing with any local Postgres on 5432
    volumes:
      - pgdata:/var/lib/postgresql/data             # persistent storage
      - ./docker/init:/docker-entrypoint-initdb.d:ro # first-start SQL scripts (read-only)
volumes:
  pgdata:                             # named volume managed by Docker
```

14.3 What happens step by step

1. **You run `docker compose up -d`**
   Docker reads `docker-compose.yml`, downloads `postgres:16` if it isn't already cached, and starts the container `pulsegate-db` in the background (`-d`).

2. **First start only: initialise the database**
   The volume `pgdata` is empty, so Postgres creates the `pulsegate` database and user from the environment variables, then runs every `.sql` file in `docker/init/`. `01-create-test-db.sql` creates `pulsegate_test`. On later starts the volume already has data, so this step is **skipped**.

3 **The backend connects through the mapped port**
`src/main/resources/application.properties` → `localhost:55432/pulsegate`. Docker forwards host port 55432 into the container's 5432. With `ddl-auto=update`, Hibernate creates or updates the three tables from the entity classes at startup.

4 **Tests use their own database**
`src/test/resources/application.properties` → `localhost:55432/pulsegate_test` with `ddl-auto=create-drop`: a fresh schema for every test run, dropped afterwards. **Tests never touch your dev data.**

14.4 Everyday commands

Command	What it does
`docker compose up -d`	Start the database (creates it on the first run)
`docker ps`	Check that `pulsegate-db` is *Up*, and see its port mapping
`docker compose logs -f db`	Follow PostgreSQL's logs
`docker compose stop`	Stop the container; **data is kept**
`docker compose down`	Remove the container; **data is still kept** in the volume
`docker compose down -v`	Remove the container **and the volume: all data is deleted** (a full reset)
`docker exec -it pulsegate-db psql -U pulsegate -d pulsegate`	Open a SQL shell inside the container

Useful queries in that shell: `\dt` (list tables), `SELECT * FROM rate_limit_policies;`, `SELECT decision, count(*) FROM request_events GROUP BY decision;`

14.5 Troubleshooting (problems actually hit during this build)

Symptom	Cause	Fix
`Connection to localhost:55432 refused` / `dockerDesktopLinuxEngine` not found	Docker Desktop isn't running (common after a reboot)	Start Docker Desktop, then `docker compose up -d`
`password authentication failed for user "pulsegate"`	An old volume was created with different credentials	Check the container's settings; if needed, `docker compose down -v` and start again
`database "pulsegate_test" does not exist`	The volume existed before the init script was added, so step 2 was skipped	`docker exec pulsegate-db psql -U pulsegate -d postgres -c "CREATE DATABASE pulsegate_test OWNER pulsegate;"`
Port 55432 already in use	Another container or service has taken it	Stop it, or change the host port in both the compose file and `application.properties`

> **Why only the database is in Docker:** it's the one dependency that's painful to install and must match across machines. A natural next step is to add `Dockerfile`s for the backend and frontend, so that `docker compose up` starts the whole stack. That's also what deployment would need.

15. Error Handling

Status	When	Source
400	Invalid fields, malformed JSON, non-numeric id	`@Valid` + GlobalExceptionHandler
401	Missing, blank, unknown or inactive API key	RateLimitInterceptor → InvalidApiKeyException
404	Policy id not found	NotFoundException
409	Duplicate policy or client name	ConflictException
429	Quota used up (+ `Retry-After`)	RateLimitInterceptor → RateLimitExceededException

16. Testing (74 Tests, 95.8% Line Coverage)

Test class	What it proves
`TokenBucketTest`	Burst, refill, cap, fractional tokens, retry time, 50 threads never overspend
`FixedWindowCounterTest`	Per-window limit, reset, the 2× boundary-burst weakness, concurrency
`RateLimiterServiceTest`	Separate quotas per client, policy and algorithm changes, atomic creation for first requests
`GatewayFlowTest`	Full flow: 200 → 429 with headers, the 401 cases, live policy update, Fixed Window
`ClientPolicyApiTest`	Validation boundaries, 404, 409, malformed JSON, policy updates
`EventsAndAnalyticsApiTest`	Totals and percentiles, filters, limit clamping, CORS
`SimulationApiTest`	Real HTTP load on a random port: exactly *capacity* allowed, every request recorded
`RequestEventRecorderTest`	A database failure doesn't change the response
`ApiKeyServiceTest` · `ConsumeResultTest` · `RepositoryTest`	Key format and hashing, Retry-After rounding, queries against real Postgres

Run them with `cd backend` then `mvn test`. The coverage report is written to `backend/target/site/jacoco/index.html`.

17. Complete Backend File Map

Package	File	Responsibility
limiter	RateLimitInterceptor	The gateway: identify, limit, record
	RateLimiterService	Per-client limiter map; atomic create and replace
	RateLimiter	Interface: `tryConsume()`, `availableTokens()`
	TokenBucket	Burst + smooth refill algorithm
	FixedWindowCounter	N requests per window algorithm
	LimitSpec	Policy settings as plain values; creates the right algorithm
	ConsumeResult	allowed, remaining, retry-after
controller	PolicyController	POST / GET / PUT `/api/policies`
	ClientController	POST / GET `/api/clients`
	ProtectedController	`/api/protected/resource` (the sample API)
	SimulationController	POST `/api/simulations`
	AnalyticsController	GET `/api/analytics/summary`
	EventController	GET / DELETE `/api/events`
	HealthController	GET `/api/health`
service	ApiKeyService	Generate, hash and prefix API keys
	PolicyService	Policy rules, create and update
	ClientService	Registration; returns the key once
	SimulationService	Concurrent load generator and statistics
	AnalyticsService	Summary from SQL aggregates
	EventService	Recent events with client names
	RequestEventRecorder	Saves decisions; never blocks the response
entity	Client · RateLimitPolicy · RequestEvent · Algorithm · Decision	JPA tables and enums
repository	Client · RateLimitPolicy · RequestEvent repositories	Queries: hash lookup, aggregates, percentiles
dto	Create/Update requests, responses, AnalyticsSummary, Simulation*	API input and output shapes
exception	GlobalExceptionHandler + 4 exception types	ProblemDetail JSON errors
config	WebConfig · OpenApiConfig	Interceptor, CORS, Swagger UI

18. End-to-End Summary

```
Admin  ──► create policy ──► register client ──► receives API key (stored only as SHA-256)
Client ──► GET /api/protected/resource  (X-API-Key)
           │
           ├─ RateLimitInterceptor: hash key → find client + policy (1 query)
           │     └─ not found ──────────────────────────► 401  (recorded)
           ├─ RateLimiterService.tryConsume()  in memory, O(1), thread-safe
           │     └─ no quota ──────────────────────────► 429 + Retry-After (recorded)
           └─ ProtectedController ─────────────────────► 200 + X-RateLimit-* (recorded)
Dashboard ◄── SQL aggregates of request_events (totals, p50/p95/p99, per client, per minute)
Simulator ──► N concurrent real HTTP requests through the same path ──► measured results
```

Explain it in 30 seconds (interview)

> "PulseGate is a rate-limiting API gateway. A Spring interceptor identifies each caller by a hashed API key, loads their policy, and asks an in-memory limiter, either Token Bucket or Fixed Window, for permission. Each check takes constant time and is made thread-safe with `synchronized` buckets and `ConcurrentHashMap.compute`. Allowed requests reach the API; the rest get a 429 with `Retry-After`. Every decision goes to Postgres for a live React dashboard, and a built-in simulator sends real concurrent traffic. That's how I measured Token Bucket smoothing traffic while Fixed Window lets it through in bursts."

Known limitations and next steps

Limitation	Next step
Quotas are in one server's memory	Redis + an atomic Lua script, so several instances share one quota
Two database round trips per request	Cache key → client + policy; write events asynchronously in batches
Admin APIs have no login	Admin authentication; protect or disable the simulator outside local development
Event table grows without limit	Retention job or time-partitioned table
Fixed Window boundary burst	Add a Sliding Window algorithm as a middle ground

Measured on Windows 11 · Intel Core i5-12450H · Java 21.0.11 · PostgreSQL 16 in Docker, with the load generator in the same JVM. The figures are for comparison, not capacity claims.